

⇒ Lab 2(b): Depth-First Search

Same tree, Same Query: A to H Now change one line of your BFS program to make it depth-first-
G: DFS visits fewer nodes here than BFS. Is DFS therefore the better Algorithm? Move the goal from under E to under G and re-run before you answer
Definition:

Depth-First Search (DFS) is a graph traversal Algorithm that explores as far as possible along one path before coming back (backtracking) to explore other paths. In simple words, DFS keeps moving deeper into the graph until it reaches a node that has no unvisited neighbors. Then it returns to the previous node and continues exploring the remaining paths.

Idea: Instead of checking every nearby path first, you chose one path and keep walking until you cannot go any further. Then you come back to the last junction and try another path.

LAB 2b: Depth-First Search

```
graph = { 'A': ['B', 'C'],  
          'B': ['D', 'E'],  
          'C': ['F', 'G'],  
          'D': [], 'E': ['H'],  
          'F': [], 'G': [], 'H': [] }
```

- The graph is represented using a dictionary.
- Each key represents a node.
- The corresponding list contains its neighboring nodes.

//_

```
def dfs(start, goal):
```

```
    stack = [start]
```

```
    visited = []
```

`def dfs(start, goal):` defines a function named `dfs` that performs the Depth-First Search (DFS) algorithm. It takes two parameters: `start` (the source node) and `goal` (the destination node).

`stack = [start]` initializes a stack with the starting node. The stack stores the complete path from the source node to the current node. Since DFS uses a stack, it follows the LIFO.

`visited = []` creates an empty list to store the nodes that have already been visited. This helps preventing revisiting the same node and avoids infinite loops during graph traversal.

- While stack:

```
    path = stack.pop()
```

The while loop runs until the stack becomes empty. `stack.pop()` removes the last path from the stack, following the LIFO.

- `node = path[-1]`

The last node in the current path is selected as the current node for exploration.

- if `node == goal`;
 return `path, visited`.

- If the current node is the goal node, the function

returns the path to the goal and the list of visited nodes.

- If node not in Visited:
visited.append(node)

If the node has not been visited before, it is added to the visited list to avoid revisiting it.

- for n in reversed(graph[node]):
stack.append(path + [n])

The neighboring nodes are added to the stack in reverse order so that the left child is explored first during DFS.

- return None, visited

If the goal node is not found after exploring all possible paths, the function returns None and the list of visited nodes.

- p, v = dfs('A', 'H')
print("DFS path :", p)
print("DFS visited:", v)

The dfs() function is called with A as the start node and H as the goal node. The returned path is stored in p and the visited nodes are stored in v.

This statement print("DFS path:", p) displays path
Statement print("DFS visited:", v) displays order of nodes

LAB 2b: Depth-First Search

```
graph = { 'A': ['B', 'C'],  
          'B': ['D', 'E'],  
          'C': ['F', 'G'],  
          'D': [], 'E': ['H'],  
          'F': [], 'G': [], 'H': [] }
```

```
def dfs(start, goal):
```

```
    stack = [[start]]
```

```
    visited = []
```

```
    while stack:
```

```
        path = stack.pop()
```

```
        node = path[-1]
```

```
        if node == goal:
```

```
            return path, visited
```

```
        if node not in visited:
```

```
            visited.append(node)
```

```
            for n in reversed(graph[node]):
```

```
                stack.append(path + [n])
```

```
    return None, visited
```

```
p, v = dfs('A', 'H')
```

```
print("DFS path :", p)
```

```
print("DFS visited:", v)
```

//_

(Output): DFS path: ['A', 'B', 'E', 'H']
DFS visited: ['A', 'B', 'D', 'E']

DFS is not always the better algorithm just because it visits fewer nodes.

When the goal node is H under E, DFS reaches it quickly by exploring the left branch first, so it visits fewer nodes than BFS.

The performance of DFS depends on the location of the goal node. Therefore, DFS is not always better than BFS. If the goal is near the root or the shortest path is required, BFS is generally the better choice.

If memory usage is limited or the goal is expected to be deep in the graph, DFS can be more suitable.